Technical report — Clausus

A masking layer, built on Apertus, that keeps confidential content out of every model call: text, slides and Swiss German audio.

- **Track:** Track 2B — Clausus
- **Event:** Online
- **Team:** Clausus — Joel Scheuner
- **Demo:** https://youtu.be/OSoTQuUp_lg

1. Summary

Where the idea comes from

The Swiss companies I talk to want two things at the same time. They want to use language models on what they work with every day: notes, documents, slides and above all meetings, which are mostly held in Swiss German. And they have to stay compliant: client names, amounts and projects must not end up with a US cloud provider, and they must be able to show where their data went. Today they have to choose. The tools that understand Swiss German audio well run on US clouds, and the ones they are allowed to use understand Swiss German poorly. Even where the privacy question is settled for one tool, the result moves on to a second tool that nobody checked. I heard this again and again in a Swiss founders' community I am part of.

Clausus is my answer to that: privacy and Swiss German in one place, built on Apertus, because Apertus is open, made in Switzerland, and runs where the company decides.

What it does

Clausus is one layer that every model call goes through. It turns the Swiss German audio into text on the company's own machine. It also reads slides and text, finds what is confidential (people, companies, projects, places, amounts, dates, contact data) and replaces each of these with a placeholder. The model that writes the answer only ever sees `[PERSON_1]`, `[ORG_2]` and `[AMOUNT_1]`. The originals stay in a vault on the company's side and are put back at the end. Every run leaves a receipt that lists each call that left Clausus, and whether it carried masked or original text. Clausus runs as its own app with a playground and an API, or as a guardrail in a LiteLLM proxy in front of the models a company already uses: any model behind an OpenAI-compatible API. If Clausus is down, the proxy lets nothing through.

Detection runs on Apertus 1.5, in five layers. On a held-out set of 24 invented business texts (191 marked spans), the first measurement leaked **2.6 %** of the spans. Microsoft Presidio leaked **51.8 %** on the same set with the same scoring. The current version leaks 0.5 % (section 5).

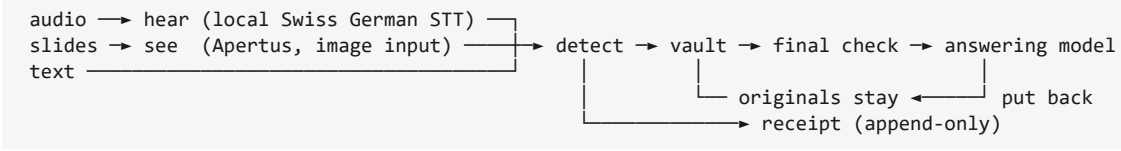
How it is used

- **Playground** (`make run`, then `localhost:8080`): drop in audio, a slide or text. Three columns show the original with every find marked, exactly what the model sees, and the answer with the real names put back. Next to it are the vault, the receipt and a chat. The playground is built for this hackathon, to make every step visible. It is a window into the system, not the product interface.
- **API**: `POST /redact` and `POST /meeting/upload` for one's own tools.
- **Guardrail**: existing apps keep their code and only point at a LiteLLM proxy with Clausus in front of the model (`docs/litellm.md`).

Why I built it this way

I measured before I polished. The first thing after a working pipeline was a held-out set and a comparison with Presidio, the common open-source tool for this job. Every detection layer exists because a measurement showed a blind spot in the others. Speech runs on a dedicated local model because Apertus's own audio input invented text in my test. And everything fails closed: when a part is unsure or down, nothing goes out.

2. Architecture



Two containers, started by `make run` (`docker-compose.yml`):

Component	Code	What it does	Runs where
`hear`	`src/hear_service/`	Swiss German speech → Standard German text with `Flix-AI/flix-swissgerman-full` (Whisper-based, Apache-2.0). It also exposes an OpenAI-style `/v1/audio/transcriptions` endpoint. The model is freed from GPU memory after 10 min idle.	Locally, on CPU or GPU
`clausus`	`src/clausus/`	FastAPI: detection, vault, router, final check, receipts, playground, chat, LiteLLM guardrail	Locally
Apertus 1.5 70B / 8B	—	Detection, judging, slide reading and the answer	An OpenAI-compatible endpoint (`LLM_BASE_URL`)

Detection (`src/clausus/detect/`) has five layers. Each model pass has blind spots the others do not share, and I measured this rather than assumed it:

1. **Rules**: regular expressions for what has a fixed form, such as phones, e-mails, IBAN, AHV numbers, amounts (including "zwöiefüfzgtuusig Franke" and "45'000" without a currency), dates, quarters ("Q4 2026"), "Ende November" and a name after "Frau/Herr".
2. **Broad pass**: Apertus 70B marks every category.
3. **Person pass**: Apertus 8B, asked only for people. The 70B model reliably missed a lone Swiss German first name ("D'Nadine"). The narrow pass finds it.
4. **Names pass**: Apertus 70B, asked only for organisations, projects and places, as a plain list.
5. **Judge**: Apertus 70B rules on finds that only one pass made. It can only *open* a find, and only when it says so clearly; an unsure or unreadable verdict keeps the placeholder. Persons are never judged, because a missed person is the worst leak. The judge reads the organisation's own rules for what may stay open ("weekdays", "roles without names", "Excel"). I added it after tracing each needless placeholder in a long text: about two thirds came from a single pass. A majority vote would have dropped those too, and with them the one real surname that only the person pass had found.

Texts longer than 400 characters are cut into sections at line ends, and the sections run in parallel. If any call fails, the whole detection fails. Half a detection is never treated as a full one (fail closed).

Vault (`vault.py`): one placeholder per entity, also for short forms that come later ("Thomas Brunner" → later "Brunner" → same `[PERSON_1]`; "Projekt Kleinhünige" → later "Kleinhünige"). An ambiguous short form ("Thomas" when two Thomases appear) is never merged. After detection, the vault sweeps the text for every further mention of a known name. This is the main reason Clausus beats Presidio on the same spans: Presidio judges each mention on its own.

Router and final check (`router.py`, `final_check.py`): code reads a rules file and decides per task whether it stays local or may go, masked, to an external model. Nothing in the content can change that. Before anything goes external, the rules run again on the masked text, and Apertus is asked whether anything is still identifiable. Any hit means the task stays local. The default sends nothing external.

Receipt (`receipt.py`): append-only, one per run. It lists every outgoing call, including the detection calls, which by design carry the original text to the detection endpoint. It stores the masked text verbatim and never the originals.

Unsourced figures. The answering model sees only placeholders, so any number, amount or date in its answer that appears in nothing it received was invented or calculated. Clausus marks such figures in the protocol, the playground and the chat instead of letting them pass as facts.

Chat and profile (`sessions.py`, `profile.py`): the playground has a chat whose vault and history are stored encrypted (AES-256-GCM, one file per session, the session id as associated data). The organisation's confidentiality profile is

encrypted the same way. It starts from default rules. Apertus can turn a written privacy policy into a proposal for a person to confirm, and each "don't mask" or "always mask" click adds to it.

Integration: `POST /redact`, `POST /meeting`, `POST /meeting/upload`, `GET /receipts/{id}`, and `POST /beta/litellm_basic_guardrail_api`. With the last one, Clausus works as a guardrail in an existing LiteLLM proxy (`docs/litellm.md`). I tested this end to end against LiteLLM 1.83.9: the model saw placeholders and the user got the originals. With Clausus stopped, the request failed with HTTP 500 and the model received nothing.

Target architecture (mandatory)

Clausus can be deployed in all three ways. The code is the same, only `LLM_BASE_URL` changes.

- **c) Sovereign Swiss cloud**: this is how I developed it. Apertus runs on the Hack Apertus endpoint, which is Swiss-hosted. I also ran the speech model on **Phoeniqs** (Swiss cloud, cluster `ai-3`) on an H100 in an OpenShift VM. There, 50 s of audio took **3.4 s** (14.7× real time; 5.3 s on the cold first run). The same file takes 224 s in my CPU container and about 14 s on a local RTX 3070. These are one run per environment, on different days.
- **a) On-premise**: both containers run on any Docker host. Apertus 8B and 70B are open weights and can be served with any OpenAI-compatible server (for example vLLM) on the organisation's own GPUs. Clausus then talks only to that host.
- **b) Air-gapped**: works once the images are built and the speech model (about 3 GB) sits in the `hf-cache` volume. At runtime Clausus needs only the Apertus endpoint inside the network. No telemetry, and no call to anything else.

Build time vs. runtime. At build time Clausus downloads Python packages and the speech model from Hugging Face on first start. At runtime it calls the Apertus endpoint, plus an external model only if a rule and a configured target both allow it (off by default).

What the detection endpoint sees. Detection has to read the original. So the endpoint that runs detection has to be trusted: Swiss-hosted or your own. Only the *answering* model is kept to placeholders. A setup where no original leaves the building at all is the on-premise one.

3. Use of Apertus

- **Models:** `swiss-ai/Apertus-v1.5-70B` (broad pass, names pass, judge, final check, slide reading, answer) and `swiss-ai/Apertus-v1.5-8B` (person pass).
- **How it is used:** inference only, with prompts in German. There is no fine-tuning.
- **Where it runs:** the Hack Apertus OpenAI-compatible endpoint (`hackapertus.livemap.sh`), `temperature: 0`. The endpoint is configured through `LLM_NAME`, `LLM_BASE_URL` and `LLM_API_KEY`.

Each model pass asks for JSON. A tolerant parser repairs common breakage: one 8B answer contained `R\uegg`, which used to fail the whole detection. On HTTP 5xx or 429 a call is retried once.

Other models.

- `Flix-AI/flix-swissgerman-full` (Apache-2.0) turns speech into text. I use it because Apertus's own audio input invented the end of my test recording and got 2 of 11 confidential spans right (2 more in part), while Flix got 8 right and 2 in part (one 50 s recording, 01.10.2026).
- Microsoft Presidio runs only as a baseline in evaluation (`eval_baselines/`).

Nothing else generates or judges content.

4. Data

All evaluation data is invented and written for this repository (`data/eval/`, CDLA-Permissive-2.0). Large public organisations (Swisscom, SBB, a cantonal bank) appear in a few cases, because real business texts mention them. IBAN and AHV numbers carry invalid check digits on purpose.

Set	Cases	Spans	Purpose	Blind?
`cases.json`	24	167	development: I fixed against it	no
`holdout.json`	24	191	written by a separate agent after the first fixes; the reported number	first run yes, later runs no
`long.json`	4	89	long texts (1.0–1.4k chars)	no: it set the section size
`traps.json`	8	113	written to break Clausus (dialect, short forms, over-masking) by an agent without code access	no: fixes were built on its leaks

Four styles of six cases each: speech-to-text output, written Swiss German (Zurich, Bern, Basel, Lucerne, Valais), slide text, and German with English business terms. The only recording is 50 s of me reading an invented text. No data from other people is used.

5. Evaluation

Metric. Leak rate = the share of marked spans that still appear in the text the answering model sees, fully or in part. A partial leak counts only where the span stood (I changed this on 08.10.: before, a shared word anywhere counted). I also count **over-masking**: spans masked although they are not secret. Over-masking makes the answer useless, and the leak rate alone hides it.

First contact, held-out set (02.10.2026, 3 runs, identical): 2.6 % leaked (5 of 191). By style: written Swiss German 5.7 %, mixed German/English 2.2 %, transcript 2.0 %, slides 0 %.

Baseline on the same set (06.10.2026). Presidio 2.2.364, `de_core_news_lg`, all German recognizers, scored with the same function:

Setup	all 191 spans	types Presidio covers (122)	Swiss German (53)	over-masked
Presidio (baseline)	51.8 %	32.8 %	52.8 %	12
Clausus (02.10.)	2.6 %	2.5 %	5.7 %	2

Presidio has no recognizer for amounts or projects, so the second column compares only the types it covers.

Current version (13.10.2026, one run each, without and with the judge; the shipped default is with):

Set	leaked, without judge	leaked, with judge	over-masked without → with
held-out, 24 short cases (191)	0.5 % (1)	0.5 % (1)	2 → 1
held-out joined into 4 long texts (191)	0.5 % (1)	1.1 % (2)	3 → 1
`long.json` (89)	0.0 %	0.0 %	5 → 3
`traps.json` (113)	0.9 % (1)	0.9 % (1)	11 → 5
`cases.json`, development (167)	0.6 % (1)	0.6 % (1)	1 → 1

The judge halves the needless placeholders and leaves the leaks where they were. The one extra leak on the joined set is a single span in a single run; I have seen differences of one span between identical runs before. Each run takes 20 s to 2 min for a whole set. Only the held-out first-contact number is blind; read the rest as the state of the code, not as a result.

Length matters. All short cases are under 400 characters. When I joined six held-out cases into one text, one detection call per text leaked 9.4 %. With 400-character sections the same spans leak 2.1 % (07.10.). The benchmark only showed this after a live test with a real 2755-character text, where amounts and a surname reached the model in clear.

Speech. The hear service writes Standard German with capital letters, which removes most dialect traps for the text layer. It adds a new class of error, **mishearing**: "Huber Treuhand AG" became "Hoover Treuhand AG", "Brunner AG" became "Bruno AG", and "Bank Zug" lost "Zug". A misheard name is still masked as a name, but the original name is no longer in the text, so the vault cannot match it later. On the H100 the transcript had the same mishearings as on my local GPU: faster, not more accurate. The transcript is not identical across environments either: in a cold-start run on CPU (13.10.) "Projekt Alpenblick" came out as "Alpublick".

Tests. 271 unit and integration tests (`make test`). On the core modules (detection, vault, guardrail) I broke the code on purpose, and each deliberate bug turned a test red.

6. Limitations

- **Small, synthetic sets.** Only the first held-out number (2.6 %) is blind; the later numbers show the state of the code. A set marked by people who did not build Clausus comes next.
- **Names only a model can find** still leak now and then: a public organisation, a place, an amount said vaguely in dialect.
- **Mishearing.** A misheard name is masked, but the vault can no longer match it later.
- **Speed on CPU.** 50 s of audio plus a slide take about 3.5 min on CPU, seconds on a GPU.
- **One user.** The prototype has no user management yet, and in the demo setup the vault key sits next to the data. For real use it belongs in the environment.

7. Reproducibility

```
git clone https://github.com/jou2512/clausus && cd clausus/track_2b
cp .env.example .env            # set LLM_API_KEY
make run                        # or: docker compose up --build   (GPU: make run-gpu)
# open http://localhost:8080
make test
uv run python -m clausus.eval --cases data/eval/holdout.json --judge
uv run python -m clausus.eval --cases data/eval/holdout.json --concat 6 --judge
```

Numbers in this report: commit `3c86bfc`, Apertus 1.5 on the Hack Apertus endpoint, `temperature: 0`. On 13.10. a fresh clone built and started in about 2 min (some base layers cached), the speech model (about 3 GB) downloaded in another 6 to 8 min, and the first upload of 50 s audio plus a text came back after 167 s on CPU, with only placeholders sent to the answering model.

8. Where it is going

Clausus is meant to become a package a company installs once: the proxy, its confidentiality profile and the receipts, with every tool it already uses connected behind it. The playground then gives way to interfaces for real use: an admin view for the profile and the receipts, and connectors for meeting and document tools.

The core of it: plugging in your own trained models. Every model Clausus uses already sits behind a configurable OpenAI-compatible endpoint (`LLM_BASE_URL`, `LLM_NAME`, and the transcription endpoint). Today that is Apertus and the Flix speech model. The next step is to make this a first-class function: a company plugs in a model trained for its own work, and Clausus uses it for detection, speech or the answer.

- **Model packages per sector.** Ready-made packages for SMEs, trust companies, medical practices and public offices: a fine-tuned Apertus detector, the sector's types and rules (what counts as confidential there), examples and a default profile. A trust company gets mandates, tax IDs and client numbers out of the box, a practice gets patients and diagnoses.
- **Training on the company's own data.** Its client and staff names, its vocabulary, its dialect, so detection and speech recognition fit that company.
- **A small local detector** trained on Apertus's decisions (teacher → student), so most texts need no model call and the expensive passes run only where it is unsure.
- **Better Swiss German speech**: the speech model fine-tuned on the company's names and dialect, plus correcting mishearings with its name list before detection.
- **A held-out set marked by people**, in several dialects.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. Code: Apache-2.0. Evaluation data: CDLA-Permissive-2.0.

References

- Apertus 1.5: https://huggingface.co/swiss-ai/Apertus-v1.5-70B
- Flix Swiss German model: https://huggingface.co/Flix-AI/flix-swissgerman-full
- Microsoft Presidio: https://github.com/microsoft/presidio

- LiteLLM guardrails: https://docs.litellm.ai/docs/proxy/guardrails/quick_start